

INFORME TÉCNICO



“NETTALCO”

**SI807-U SISTEMA DE INTELIGENCIA DE
NEGOCIOS**

Grupo 5

Integrantes:

- | | |
|------------------------------------|-----------|
| • Grijalva Parra, Francisco Leonel | 20220071I |
| • Loayza Segura, Roger Salvador | 20220018K |
| • Otero Vicente, Daniel Mauricio | 20220288H |

Contenido

1.	Introducción	4
2.	Arquitectura Cloud Definida	4
2.1.	Justificación de la Nube Pública (GCP)	4
2.2.	Arquitectura General	4
3.	Servicios Cloud Utilizados	6
3.1.	Servicios utilizados en PC3	6
3.2.	Servicios utilizados en PC4	6
4.	Diseño del Data Lake (Bronce – Plata – Oro)	7
4.1.	Zona Bronce (Raw Layer)	7
4.2.	Zona Plata (Processed Layer)	8
4.3.	Zona Oro (Analytics Layer)	8
5.	Modelo Dimensional Implementado	9
5.1.	Tipo de Modelo	10
5.2.	Tablas de Hechos	10
5.2.1.	Hechos de Ventas (Fact_Ventas)	10
5.2.2.	Hechos de Producción (Fact_Produccion)	11
5.2.3.	Hechos de Inventario (Fact_Inventario)	11
5.3.	Tablas de Dimensión	12
5.3.1.	Dim_Tiempo	12
5.3.2.	Dim_Cliente	12
5.3.3.	Dim_Producto	13
5.3.4.	Dim_Empleado	13
5.3.5.	Dim_Máquina	13
5.4.	Justificación Técnica del Modelo Dimensional	14
6.	Procesos ETL / ELT Aplicados	14
6.1.	Extracción	14
6.2.	Transformación	16
6.3.	Carga a Data lake(refined/curated) y Data warehouse(BigQuery)	17
6.3.1.	Data lake(refined/curated)	17
6.3.2.	Data warehouse(BigQuery)	21
6.4.	Automatización	23
6.5.	Control de errores y monitoreo	28
6.6.	Escalabilidad Horizontal	29
7.	Scripts Utilizados	30
7.1.	Script Principal PySpark	30
8.	Consultas SQL Analíticas	30
8.1.	Validación de Volumen Total de Producción	30

8.2.	Análisis de Clientes con Mayor Volumen de Ventas	31
8.3.	Validación por Franja Horaria	32
8.4.	Identificación de Productos Más Vendidos	32
8.5.	Validación de Eficiencia Operativa	33
8.6.	Análisis de Tendencias con Promedio Móvil	33
8.7.	Análisis de Comportamiento del Cliente	34
8.8.	Rol de las Consultas SQL dentro de la Arquitectura	35
9.	Visualizaciones Finales	35
9.1.	Herramienta	35
9.2.	Dashboards Implementados	35
10.	Uso del Repositorio GitHub	40
10.1.	Estrategia de Ramas	40
10.2.	Pull Requests y Merges	40
11.	Conclusión	40

1. Introducción

El presente informe técnico describe de manera integrada la arquitectura Cloud, los servicios utilizados, el diseño del Data Lake, el modelo dimensional, los procesos ETL/ELT, los scripts desarrollados, las consultas SQL analíticas, las visualizaciones finales y la estrategia de control de versiones mediante GitHub, implementados para el proyecto de análisis de datos de Nettelco S.A.

El objetivo principal del proyecto fue diseñar e implementar una plataforma de analítica escalable, capaz de procesar grandes volúmenes de datos operativos de producción, ventas y calidad, transformarlos en información estructurada y disponibilizarlos para análisis estratégico mediante dashboards interactivos.

2. Arquitectura Cloud Definida

2.1. Justificación de la Nube Pública (GCP)

Se seleccionó Google Cloud Platform (GCP) debido a las siguientes razones técnicas:

- Integración nativa entre Cloud Storage, Dataproc y BigQuery
- Escalabilidad horizontal bajo demanda
- Separación clara entre almacenamiento, procesamiento y consumo
- Optimización de costos mediante procesamiento batch
- Alta compatibilidad con Apache Spark y herramientas BI (Looker Studio)

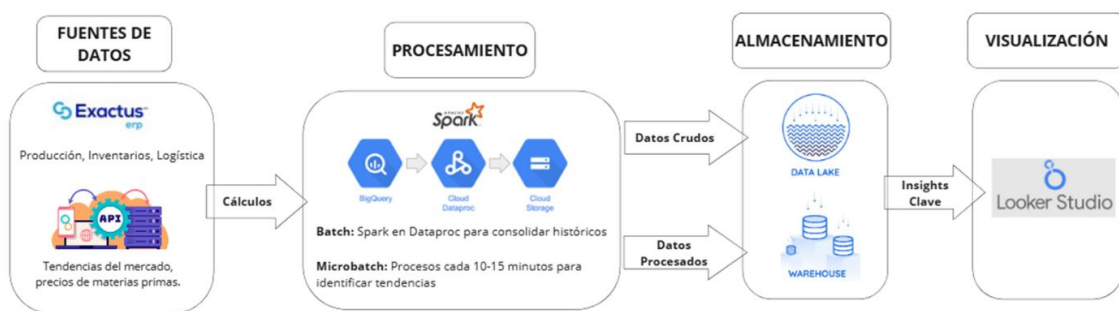
Esta arquitectura sigue el principio decoupled architecture, donde cada capa puede escalar independientemente.

2.2. Arquitectura General

Capas de la arquitectura:

- Ingesta de datos
 - Archivos CSV desde sistemas operativos (ERP, producción, control de calidad)
 - Almacenados inicialmente en Cloud Storage
- Almacenamiento

- Data Lake en Google Cloud Storage
- Data Warehouse en BigQuery
- Procesamiento
 - Apache Spark sobre Dataproc (ETL batch)
- Consumo
 - Looker Studio para visualización
 - Consultas SQL analíticas en BigQuery



Componente	Descripción
Servicios Básicos	Compute Engine, Cloud Storage, BigQuery
Servicios Avanzados	Dataproc (Batch), Cloud Functions (microprocesos), Looker Studio (visualización)
Data Lake	Buckets: raw/, trusted/, refined/
Procesamiento	Spark en Dataproc (Batch y Microbatch)
Escalabilidad	Autoscaling en Dataproc, clúster ajustable
Alta Disponibilidad & DR	Multi-Zona y replicación de datos

Seguridad	Cloud Identity, IAM granular y Secret Manager
Monitoreo & Alertas	Stackdriver / Cloud Monitoring

3. Servicios Cloud Utilizados

3.1. Servicios utilizados en PC3

Servicio	Uso técnico
Google Cloud Storage	Almacenamiento de datos crudos (CSV)
BigQuery	Consultas analíticas y Data Warehouse
Looker Studio	Visualización de KPIs
Cloud Shell	Administración y ejecución de comandos
Dataproc	Procesamiento distribuido con Spark

Justificación:

PC3 se centró en la ingesta, almacenamiento y visualización, permitiendo análisis descriptivo directo sobre BigQuery.

3.2. Servicios utilizados en PC4

Servicio	Uso técnico
Google Cloud Storage	Manejo del todo Data lake (raw/, trusted/, refined/) multizonal
Compute Engine	Infraestructura del clúster
Apache Spark	Transformaciones ETL

BigQuery	Persistencia analítica
Cloud Identity	Manejo de la seguridad de la solución en la nube
Cloud Monitoring y Logging	Visualización del log y monitoreo de los procesos de batch y microbatch,
Cloud Functions	Ejecutar los microbatchs para el data lake.
Cloud Run	Programación para los procesos del batch a las 23:00 y microbatch a demanda
Pub/Sub	Creación del trigger para la detección de eventos para el microbatch

Justificación:

PC4 introduce procesamiento distribuido, necesario para:

- Escalabilidad
- Transformaciones complejas
- Cálculo de métricas agregadas
- Creación de un entorno Big Data real
- Seguridad

4. Diseño del Data Lake (Bronce – Plata – Oro)

4.1. Zona Bronce (Raw Layer)

Ubicación:

gs://nettalco-data-bd_grupo05/raw

Características técnicas:

- Datos en formato CSV
- Sin transformación

- Representación fiel del origen
- Esquema flexible

Justificación:

Permite:

- Auditoría de datos
- Reprocesamiento
- Punto de inicio para la trazabilidad completa

4.2. Zona Plata (Processed Layer)

Ubicación:

gs://nettalco-data-bd_grupo05/trusted

Características técnicas:

- Datos limpios
- Tipos de datos inferidos
- Normalización básica
- Agregaciones iniciales

Justificación:

Optimiza los datos para consumo analítico sin perder granularidad relevante.

4.3. Zona Oro (Analytics Layer)

Ubicación:

gs://nettalco-data-bd_grupo05/refined

Características técnicas:

- Datos altamente refinados y consolidados
- Resultado final de los procesos ETL en Spark
- Estructura orientada a consumo analítico
- Esquemas definidos (schema-on-write)

- Datos agregados y optimizados
- Persistencia en formatos eficientes (Parquet / CSV optimizado)
- Fuente única y confiable para capas de consumo

Justificación:

Permite:

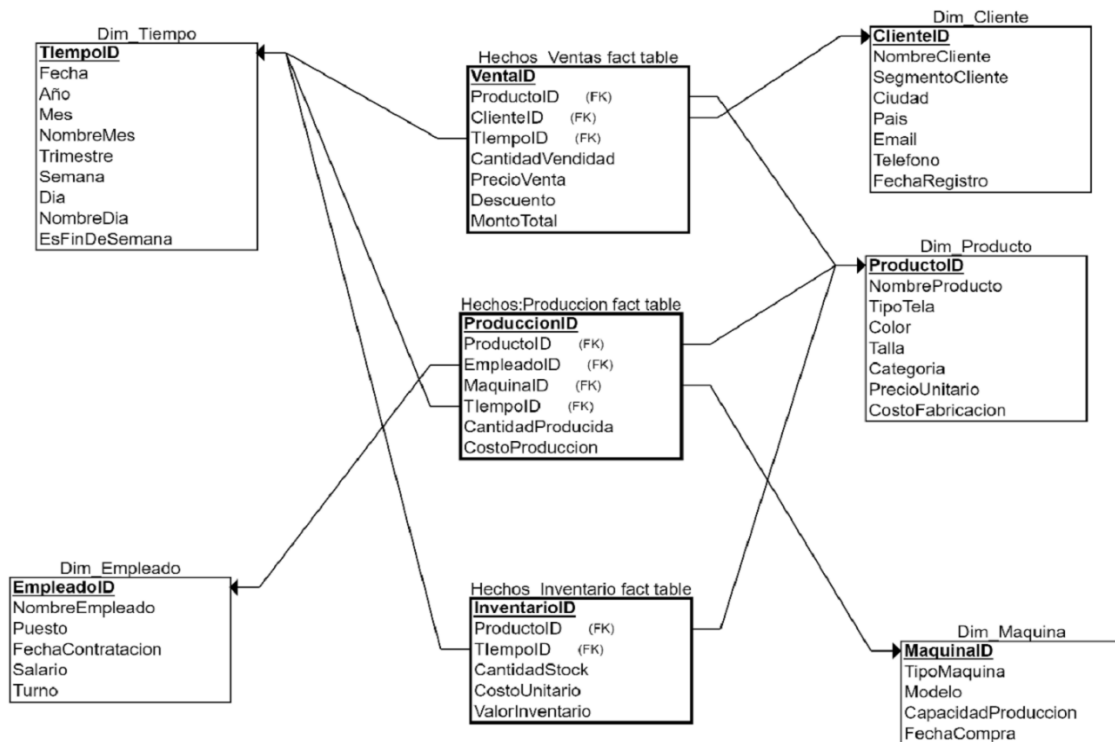
- Separar claramente los datos operativos de los analíticos (listo para reportes)
- Evitar reprocesamientos innecesarios
- Reducir la carga computacional en etapas posteriores
- Mantener independencia entre almacenamiento y consumo
- Facilitar la gobernanza y trazabilidad del dato

Esta zona no reemplaza al Data Warehouse, sino que lo alimenta.

5. Modelo Dimensional Implementado

El modelo dimensional implementado se diseñó con el objetivo de optimizar el análisis analítico, facilitar la generación de indicadores clave y reducir la complejidad de las consultas SQL utilizadas por las herramientas de Business Intelligence. El diseño se basa en los principios clásicos de modelado dimensional de Kimball, adaptados al contexto operativo de Nettalco.

El diagrama presentado muestra una arquitectura dimensional clara, con tablas de hechos bien definidas y dimensiones compartidas, permitiendo análisis cruzados entre ventas, producción e inventario.



5.1. Tipo de Modelo

Se implementó un modelo dimensional tipo estrella (Star Schema), donde:

- Las tablas de hechos concentran las métricas cuantitativas del negocio.
- Las tablas de dimensión contienen los atributos descriptivos utilizados para segmentación y análisis.
- Las relaciones son muchos-a-uno desde las tablas de hechos hacia las dimensiones.

Justificación técnica del Star Schema:

- Simplifica las consultas analíticas
- Reduce el número de joins complejos
- Mejora el rendimiento en motores columnar como BigQuery
- Es el modelo óptimo para herramientas BI como Looker Studio

5.2. Tablas de Hechos

Tal como se observa en el diagrama, el modelo incluye múltiples tablas de hechos, cada una orientada a un proceso clave del negocio.

5.2.1. Hechos de Ventas (Fact_Ventas)

Claves foráneas:

- ProductoID
- ClienteID

- TiempoID

Métricas principales:

- CantidadVendida
- PrecioVenta
- Descuento
- MontoTotal

Rol analítico:

Permite analizar el comportamiento de ventas por producto, cliente y tiempo, soportando indicadores comerciales y financieros.

5.2.2. Hechos de Producción (Fact_Produccion)**Claves foráneas:**

- ProductoID
- EmpleadoID
- MaquinaID
- TiempoID

Métricas principales:

- CantidadProducida
- CostoProduccion

Rol analítico:

Soporta el análisis de eficiencia operativa, productividad por máquina y rendimiento del personal.

5.2.3. Hechos de Inventario (Fact_Inventario)**Claves foráneas:**

- ProductoID
- TiempoID

Métricas principales:

- CantidadStock
- CostoUnitario
- ValorInventario

Rol analítico:

Permite monitorear niveles de stock, valorización de inventarios y rotación de productos.

Nota importante:

Aunque en la implementación analítica se utilizan tablas consolidadas (por ejemplo Fact_Produccion_Ventas), el diseño conceptual mantiene la separación lógica de procesos, lo cual es una buena práctica de modelado dimensional.

5.3. Tablas de Dimensión

Las tablas de dimensión proporcionan el contexto descriptivo necesario para el análisis multidimensional. Estas dimensiones son compartidas entre las distintas tablas de hechos, favoreciendo la consistencia analítica.

5.3.1. Dim_Tiempo

Atributos:

- Fecha
- Año
- Mes
- NombreMes
- Trimestre
- Semana
- Día
- NombreDia
- EsFinDeSemana

Justificación técnica:

Permite análisis temporales detallados, comparaciones interanuales y agregaciones por distintos niveles de granularidad.

5.3.2. Dim_Cliente

Atributos:

- ClienteID
- NombreCliente
- SegmentoCliente
- Ciudad
- País
- Email
- Teléfono
- FechaRegistro

Justificación técnica:

Facilita segmentación de clientes, análisis de comportamiento y estudios de fidelización.

5.3.3. Dim_Producto**Atributos:**

- ProductoID
- NombreProducto
- TipoTela
- Color
- Talla
- Categoría
- PrecioUnitario
- CostoFabricación

Justificación técnica:

Soporta análisis de rentabilidad por producto, estilos más vendidos y control de costos.

5.3.4. Dim_Empleado**Atributos:**

- EmpleadoID
- NombreEmpleado
- Puesto
- FechaContratación
- Salario
- Turno

Justificación técnica:

Permite evaluar productividad y eficiencia del recurso humano.

5.3.5. Dim_Máquina**Atributos:**

- MaquinaID
- TipoMaquina
- Modelo
- CapacidadProducción

- FechaCompra

Justificación técnica:

Facilita análisis de desempeño operativo y mantenimiento.

5.4. Justificación Técnica del Modelo Dimensional

El modelo dimensional implementado presenta las siguientes ventajas técnicas:

- Reducción de joins complejos, favoreciendo consultas más simples
- Alto rendimiento analítico en motores columnar
- Flexibilidad para agregar nuevas métricas
- Consistencia entre procesos de ventas, producción e inventario
- Compatibilidad directa con herramientas BI

Este diseño garantiza que la plataforma analítica sea escalable, mantenible y orientada a la toma de decisiones.

6. Procesos ETL / ELT Aplicados

Flujo ETL: Extract → Transform → Load

6.1.Extracción

- Datos internos: ERP Exactus, producción y logística.
- Lectura de CSVs desde Cloud Storage (raw/) con PySpark:

```
# procesamiento_nettalco_etl.py
```

```
from pyspark.sql import SparkSession
```

```
from pyspark.sql.functions import col, sum as _sum, avg, count, round, when,
to_timestamp, hour
```

```
from pyspark.sql.window import Window
```

```
from pyspark.sql.types import IntegerType
```

```
import logging
```

```
import os
```

```
import sys
```

```
# -----
```

```
# Logging (esto va a stdout -> Cloud Logging)
```

```
# -----
```

```

logging.basicConfig(stream=sys.stdout, level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")

logger = logging.getLogger("nettalcoproc")

logger.info("Iniciando job PySpark - Nettelco ETL")

# -----
# Configuración de Spark
# -----

spark = SparkSession.builder \

    .appName("Nettalco Big Data Processing - ETL") \

    .config("spark.sql.legacy.timeParserPolicy", "LEGACY") \

    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

logger.info("SparkSession creada")

# -----
# Rutas (raw -> trusted -> refined)
# -----

PROJECT_BUCKET = os.environ.get("PROJECT_BUCKET", "nettalco-data-
bd_grupo05")

RAW_PREFIX = f"gs://{PROJECT_BUCKET}/raw/"

TRUSTED_PREFIX = f"gs://{PROJECT_BUCKET}/trusted/"

REFINED_PREFIX = f"gs://{PROJECT_BUCKET}/refined/curated/"

detalle_path = f"{RAW_PREFIX}detalle-produccion-costura.csv"

cliente_path = f"{RAW_PREFIX}produccion-costura-cliente.csv"

linea_path = f"{RAW_PREFIX}produccion-costura-linea-cliente.csv"

segundas_path = f"{RAW_PREFIX}segundas-prendas.csv"

```

```
logger.info("Rutas establecidas: RAW=%s, TRUSTED=%s, REFINED=%s",
RAW_PREFIX, TRUSTED_PREFIX, REFINED_PREFIX)
```

```
# -----
```

```
# Cargar los datos (raw)
```

```
# -----
```

```
logger.info("Leyendo archivos raw...")
```

```
detalle_produccion = spark.read.csv(detalle_path, header=True, inferSchema=True)
```

```
produccion_costura_cliente = spark.read.csv(cliente_path, header=True,
inferSchema=True)
```

```
produccion_costura_linea_client = spark.read.csv(linea_path, header=True,
inferSchema=True)
```

```
segundas_prendas = spark.read.csv(segundas_path, header=True, inferSchema=True)
```

```
logger.info("Lectura raw completa: detalle=%s, cliente=%s, linea=%s, segundas=%s",
```

```
detalle_produccion.count(), produccion_costura_cliente.count(),
```

```
produccion_costura_linea_client.count(), segundas_prendas.count())
```

6.2.Transformación

- Limpieza y tipificación de columnas.
- Conversión de fechas a timestamp.
- Guardado intermedio en formato **parquet** en trusted/.

```
# -----
```

```
# Data typing / limpieza mínima -> trusted
```

```
# -----
```

```
logger.info("Transformando datos y escribiendo trusted...")
```

```
detalle_produccion = detalle_produccion.withColumn("PRENDAS",
col("PRENDAS").cast(IntegerType()))
```

```
produccion_costura_cliente = produccion_costura_cliente.withColumn("PRENDAS",
col("PRENDAS").cast(IntegerType()))
```



```

produccion_costura_linea_client                                     =
produccion_costura_linea_client.withColumn("PRENDAS",
col("PRENDAS").cast(IntegerType()))

```

```

segundas_prendas          =      segundas_prendas.withColumnn("FALLAS_SEGUNDAS",
col("FALLAS_SEGUNDAS").cast(IntegerType())) \

                                .withColumnn("INSPECCION_TOTAL",
col("INSPECCION_TOTAL").cast(IntegerType()))

```

Normalizar fecha a columna timestamp

```

detalle_produccion = detalle_produccion.withColumn(

    "FECHA_TERMINO_TS",

    to_timestamp("FECHA_TERMINO", "dd/MM/yyyy HH:mm:ss")

)

```

Guardar copias "trusted" (formato parquet recomendado para downstream)

```

trusted_detalle = f"{TRUSTED_PREFIX}detalle-produccion-costura/"
trusted_cliente = f"{TRUSTED_PREFIX}produccion-costura-cliente/"
trusted_linea = f"{TRUSTED_PREFIX}produccion-costura-linea-cliente/"
trusted_segundas = f"{TRUSTED_PREFIX}segundas-prendas/"

```

```

detalle_produccion.write.mode("overwrite").parquet(trusted_detalle)
produccion_costura_cliente.write.mode("overwrite").parquet(trusted_cliente)
produccion_costura_linea_client.write.mode("overwrite").parquet(trusted_linea)
segundas_prendas.write.mode("overwrite").parquet(trusted_segundas)

logger.info("Trusted escritos en parquet en: %s, %s, %s, %s",

```

```

    trusted_detalle, trusted_cliente, trusted_linea, trusted_segundas)

```

- Cálculo de métricas y datasets refinados (refined/curated): total prendas por talla, ventas por cliente, fecha de ventas, tendencias horarias, productos más vendidos, eficiencia operativa, índice ventas cliente, predicción de ventas y comportamiento de clientes.

6.3.Carga a Data lake(refined/curated) y Data warehouse(BigQuery)

6.3.1. Data lake(refined/curated)

- Uso de **Cloud Functions** + **Pub/Sub** para carga a demanda (cuando llegue un nuevo reporte csv al raw).
- Uso de **Cloud Login** para las notificaciones y seguimiento del dataproc.

```
# -----
```

```
# Procesos analíticos -> refined (curated outputs)
```

```
# -----
```

```
logger.info("Generando datasets refinados (refined)...")
```

```
# Total prendas por talla
```

```
detalle_produccion.groupBy("TALLA") \
    .agg(_sum("PRENDAS").alias("TOTAL_PRENDAS")) \
    .orderBy("TALLA") \
    .write.mode("overwrite").option("header",
    "true").csv(f'{REFINED_PREFIX}total_prendas_por_talla')
```

```
# Volumen de ventas por cliente
```

```
produccion_costura_cliente.groupBy("TCODICLIE") \
    .agg(_sum("PRENDAS").alias("TOTAL_PRENDAS")) \
    .orderBy("TCODICLIE") \
    .write.mode("overwrite").option("header",
    "true").csv(f'{REFINED_PREFIX}volumen_ventas_por_cliente')
```

```
# Fecha ventas
```

```
produccion_costura_cliente.groupBy("FECHA") \
    .agg(_sum("PRENDAS").alias("TOTAL_PRENDAS")) \
    .orderBy("FECHA") \
    .write.mode("overwrite").option("header",
    "true").csv(f'{REFINED_PREFIX}fecha_ventas')
```

```
# Tendencias por franja horaria
```

```
detalle_produccion = detalle_produccion.withColumn("HORA",
    hour("FECHA_TERMINO_TS"))
```

```
tendencias = detalle_produccion.withColumn(
    "FRANJA_HORARIA",
    when((col("HORA") >= 6) & (col("HORA") <= 11), "Mañana")
    .when((col("HORA") >= 12) & (col("HORA") <= 17), "Tarde")
    .when((col("HORA") >= 18) & (col("HORA") <= 23), "Noche")
    .otherwise("Madrugada")
).groupBy("ORDEN_PRODUCCION", "FRANJA_HORARIA") \
    .agg(count("*").alias("TRANSACCIONES"),
    _sum("PRENDAS").alias("TOTAL_PRENDAS")) \
    .orderBy("ORDEN_PRODUCCION", "FRANJA_HORARIA")

tendencias.write.mode("overwrite").option("header",
"true").csv(f"{REFINED_PREFIX}tendencias_ventas_por_franja_horaria")
```

Productos más vendidos

```
detalle_produccion.groupBy("ESTILO") \
    .agg(_sum("PRENDAS").alias("TOTAL_PRENDAS")) \
    .orderBy("ESTILO") \
    .write.mode("overwrite").option("header",
"true").csv(f"{REFINED_PREFIX}productos_mas_vendidos")
```

Eficiencia operativa

```
segundas_prendas.groupBy("FECHA") \
    .agg(
        round(
            (1 - (_sum("FALLAS_SEGUNDAS") / _sum("INSPECCION_TOTAL"))) * 100,
2
        ).alias("EFICIENCIA_PORCENTUAL")
    ) \
    .orderBy("FECHA") \
    .write.mode("overwrite").option("header",
"true").csv(f"{REFINED_PREFIX}eficiencia_operativa")
```

```
# Índice de ventas por cliente y línea
```

```
produccion_costura_linea_client.groupBy("TCODICLIE", "LINEA") \  
  .agg(_sum("PRENDAS").alias("TOTAL_PRENDAS")) \  
  .orderBy("TCODICLIE") \  
  .write.mode("overwrite").option("header",  
"true").csv(f"{REFINED_PREFIX}indice_ventas_cliente")
```

```
# Predicción de ventas (promedio móvil 7 días)
```

```
detalle_produccion.groupBy("FECHA_TERMINO_TS", "ESTILO") \  
  .agg(_sum("PRENDAS").alias("TOTAL_PRENDAS")) \  
  .withColumn(  
    "PROMEDIO_MOVIL",  
  
    avg("TOTAL_PRENDAS").over(Window.partitionBy("ESTILO").orderBy("FECHA_  
TERMINO_TS").rowsBetween(-6, 0))  
  ) \  
  .write.mode("overwrite").option("header",  
"true").csv(f"{REFINED_PREFIX}prediccion_ventas")
```

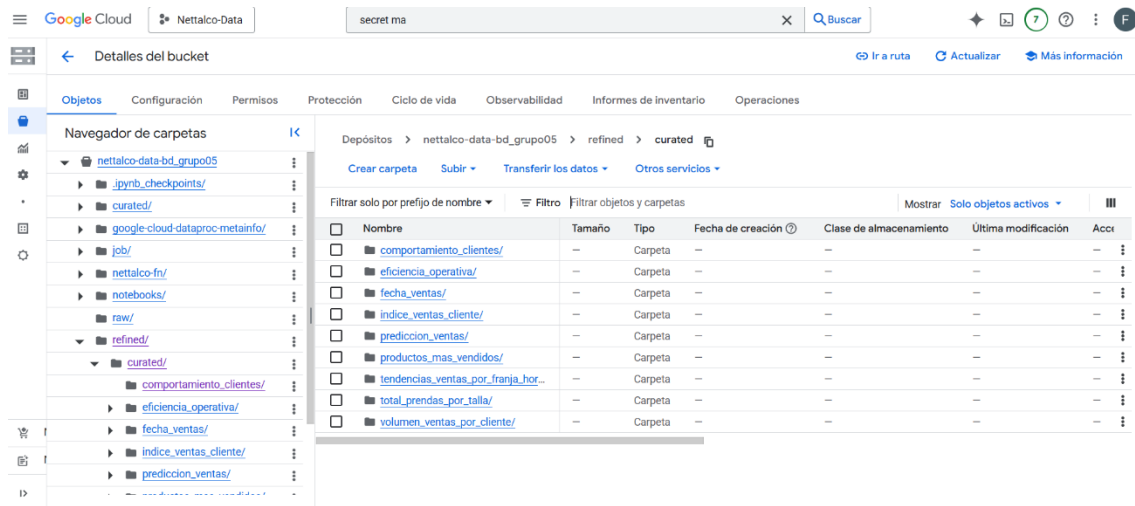
```
# Comportamiento de clientes
```

```
produccion_costura_cliente.groupBy("TCODICLIE") \  
  .agg(count("FECHA").alias("FRECUENCIA_COMPRA"),  
avg("PRENDAS").alias("PROMEDIO_PRENDAS")) \  
  .orderBy("TCODICLIE") \  
  .write.mode("overwrite").option("header",  
"true").csv(f"{REFINED_PREFIX}comportamiento_clientes")
```

```
logger.info("Refined outputs escritos en %s", REFINED_PREFIX)
```

```
logger.info("Job finalizado correctamente")
```

```
spark.stop()
```



6.3.2. Data warehouse(BigQuery)

- Uso de **Cloud Functions** + **Cloud Scheduler** para carga diaria.
- Tabla de logs en BigQuery para registrar actualizaciones.

```
import functions_framework
```

```
from google.cloud import bigquery
```

```
from datetime import datetime
```

Configuración de tablas y carpetas

```
TABLES_TO_LOAD = {
```

```
    "total_prendas_por_talla": "ventas_nettalco.total_prendas_por_talla",
```

```
    "volumen_ventas_por_cliente": "ventas_nettalco.volumen_ventas_por_cliente",
```

```
    "fecha_ventas": "ventas_nettalco.fecha_ventas",
```

```
    "tendencias_ventas_por_franja_horaria":
```

```
    "ventas_nettalco.tendencias_ventas_por_franja_horaria",
```

```
    "productos_mas_vendidos": "ventas_nettalco.productos_mas_vendidos",
```

```
    "eficiencia_operativa": "ventas_nettalco.eficiencia_operativa",
```

```
    "indice_ventas_cliente": "ventas_nettalco.indice_ventas_cliente",
```

```
    "prediccion_ventas": "ventas_nettalco.prediccion_ventas",
```

```
    "comportamiento_clientes": "ventas_nettalco.comportamiento_clientes"
```

```
}
```

```
BUCKET = "nettalco-data-bd_grupo05"
```

```
PREFIX = "refined/curated"
```

```
client = bigquery.Client()
```

```
SCRIPT_NAME = "daily_bq_update.py" # Para registrar en log
```

```
def load_csv_to_bq(table_name, gcs_path):
```

```
    """
```

```
    Carga CSV desde GCS a BigQuery y reemplaza datos existentes.
```

```
    """
```

```
    job_config = bigquery.LoadJobConfig(
```

```
        source_format=bigquery.SourceFormat.CSV,
```

```
        autodetect=True,
```

```
        write_disposition="WRITE_TRUNCATE" # reemplaza datos existentes
```

```
    )
```

```
    load_job = client.load_table_from_uri(
```

```
        gcs_path,
```

```
        table_name,
```

```
        job_config=job_config
```

```
    )
```

```
    load_job.result() # espera a que termine
```

```
    print(f'Cargado {gcs_path} a {table_name}')
```

```
    log_update(table_name)
```

```
def log_update(table_name):
```

```
    table_id = "ventas_nettalco.log"
```

```
    rows_to_insert = [
```

```
        {
```

```
            "tabla": table_name,
```

```
            "fecha_actualizacion": datetime.utcnow().isoformat(), # convierte a string
```

```
            "fuente": SCRIPT_NAME
```

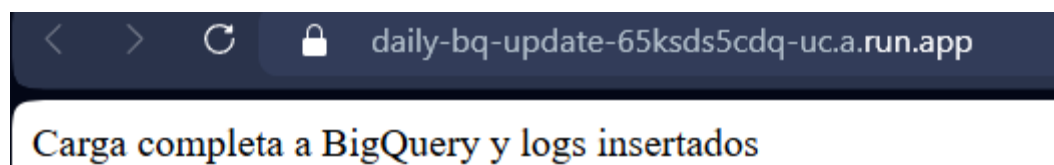
```

    }
]
errors = client.insert_rows_json(table_id, rows_to_insert)
if errors:
    print(f'Error insertando log para {table_name}: {errors}')
else:
    print(f'Log insertado correctamente para {table_name}')

@functions_framework.http
def load_refined_to_bq(request):
    """
    Función HTTP que recorre todas las carpetas de refined/curated y carga CSV a
    BigQuery.

    Luego inserta un log por cada tabla cargada.
    """
    try:
        for folder, table in TABLES_TO_LOAD.items():
            gcs_path = f'gs://{BUCKET}/{PREFIX}/{folder}/*.csv'
            print(f'Cargando {gcs_path} a {table}')
            load_csv_to_bq(table, gcs_path)
        return "Carga completa a BigQuery y logs insertados", 200
    except Exception as e:
        print(f'Error ejecutando load_refined_to_bq: {e}')
        return f'Error en la carga: {e}', 500

```



6.4. Automatización

- **Cloud Function:** La cual está dentro de cloud run, se crean las funciones que llamarán para correr los batch's-

Para refined:

```
cd ~/nettalco-fn
```

```
gcloud functions deploy nettalco-dataproc-raw-trigger \
```

```
--runtime python311 \
```

```
--trigger-topic nettalco-raw-topic \
```

```
--region us-central1 \
```

```
--timeout 540s \
```

```
--memory 1024MB \
```

```
--entry-point trigger_dataproc \
```

```
--source ./
```

Para Bigquery:

```
cd ~/nettalco-bq
```

```
gcloud functions deploy daily-bq-update \
```

```
--runtime python311 \
```

```
--trigger-http \
```

```
--allow-unauthenticated \
```

```
--region us-central1 \
```

```
--timeout 540s \
```

```
--memory 1024MB \
```

```
--entry-point load_refined_to_bq \
```

```
--source ./
```

Google Cloud | Nettalco-Data | run fun | Buscar

Cloud Run | Servicios | Implementar contenedor | Conectar repo | Escribir una función | Actualizar

Descripción general | **Servicios** | Trabajos | Grupos de trabajadores | Asignaciones de dominios

Cada servicio expone un extremo único y ajusta automáticamente la escala de la infraestructura subyacente para controlar las solicitudes entrantes. Implementa una imagen de contenedor, código fuente o función para crear un servicio.

Filtro	Nombre	Tipo de implementación	Solicitudes/seg	Región	Autenticación	Ingress	Última implementación
	daily-bq-update	(-) Función	0	us-central1	Permitir sin autenticación	Todas	hace 10 horas
	nettalco-dataproc-raw-trigger	(-) Función	0	us-central1	Necesita autenticación	Todas	hace 12 horas
	nettalco-dataproc-trigger	(-) Función	0	us-central1	Permitir sin autenticación	Todas	hace 17 horas

Notas de versión

- **Cloud Run:** Procesamiento ETL a demanda cuando se agregan archivos a raw/.

```
import functions_framework

import json

import base64

from google.cloud import secretmanager

from google.cloud import dataproc_v1


def load_config():
    """
    Carga la configuración desde Secret Manager
    """
    client = secretmanager.SecretManagerServiceClient()
    name = "projects/467475048380/secrets/nettalco_config/versions/latest"
    response = client.access_secret_version(request={"name": name})
    return json.loads(response.payload.data.decode("utf-8"))


@functions_framework.cloud_event
def trigger_dataproc(cloud_event):
    """
    Cloud Function 2nd gen disparada por Pub/Sub cuando llega un archivo a raw/.

    Lanza un Job en Dataproc usando la configuración desde Secret Manager.
    """
    try:
        # Leer configuración
        config = load_config()

        project_id = config["PROJECT_ID"]

        region = config["REGION"]

        cluster_name = config["CLUSTER_NAME"]

        pyspark_file = config["PYSPARK_FILE"]
```

```

# Extraer mensaje de Pub/Sub

pubsub_message = cloud_event.data.get("message")

if not pubsub_message or "data" not in pubsub_message:

    print("No se encontró el mensaje de Pub/Sub o 'data' vacío. Ignorando.")

    return


# Decodificar base64 y parsear JSON

payload = base64.b64decode(pubsub_message["data"]).decode("utf-8")

event_data = json.loads(payload)


# Obtener el nombre real del archivo

file_name = event_data.get("name")

if not file_name:

    print("No se encontró el nombre del archivo en el evento. Ignorando.")

    return


# Verificar que esté en raw/

if "raw/" not in file_name:

    print(f"Ignorando archivo que no está en raw/: {file_name}")

    return


print(f"Nuevo archivo detectado en raw/: {file_name}")


# Crear cliente Dataproc

job_client = dataproc_v1.JobControllerClient(

    client_options={"api_endpoint": f"{region}-dataproc.googleapis.com:443"}

)


# Configurar Job PySpark

job = {

```

```

    "placement": {"cluster_name": cluster_name},
    "pyspark_job": {
        "main_python_file_uri": pyspark_file,
        "args": [f'gs://{project_id}-bd_grupo05/{file_name}']
    },
}

```

```

# Enviar Job a Dataproc

```

```

response = job_client.submit_job(
    project_id=project_id,
    region=region,
    job=job
)

print(f'Dataproc Job lanzado correctamente: {response.reference.job_id}')

```

```

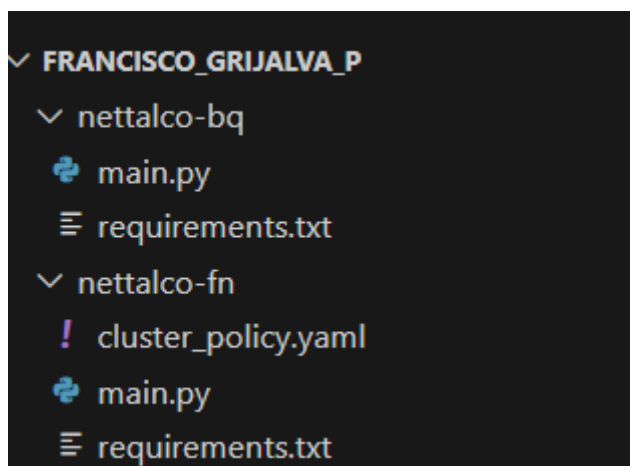
except Exception as e:

```

```

    print(f'Error al lanzar Dataproc Job: {e}')
    raise

```



- **Pub/Sub:** Trigger para lanzar jobs Dataproc.

gcloud pubsub topics create nettalco-raw-topic

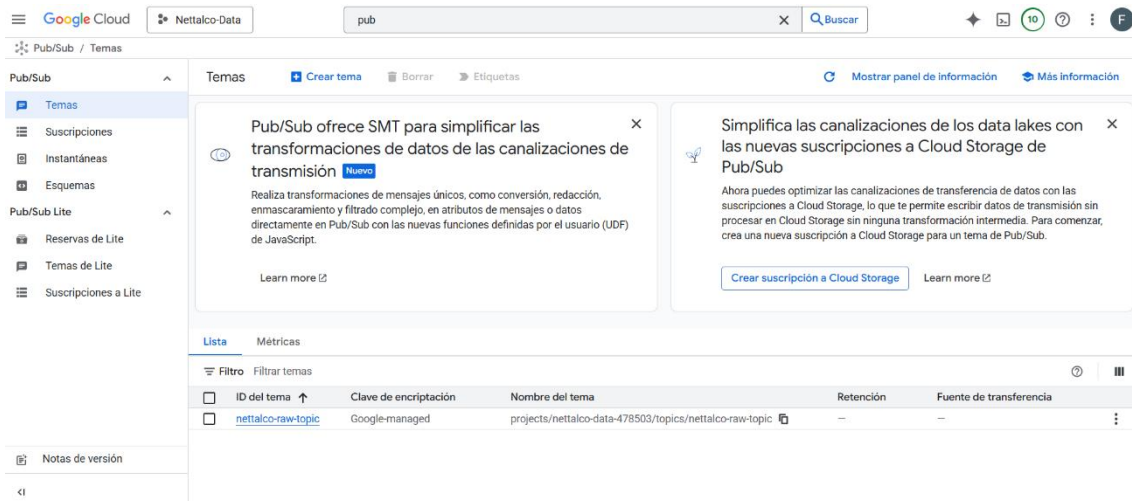
gsutil notification create \

-t nettalco-raw-topic \

-f json \

-p raw/\

gs://nettalco-data-bd_grupo05



- **Cloud Scheduler:** Job diario daily-bq-load para actualizar BigQuery.

gcloud scheduler jobs create http daily-bq-load \

--schedule "0 0 * * *" \

--uri "https://us-central1-nettalco-data-478503.cloudfunctions.net/daily-bq-update" \

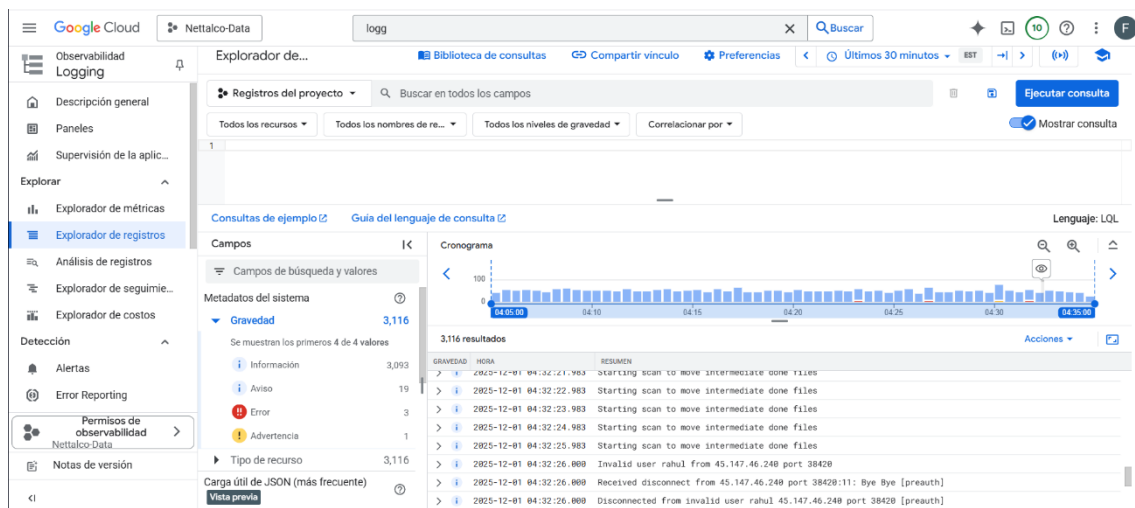
--http-method GET \

--location us-central1



6.5. Control de errores y monitoreo

- **Cloud Logging:** Registro de ejecución de jobs ETL y Cloud Functions.



- Validaciones implementadas en PySpark y logs de BigQuery.

Se crea una tabla log en el conjunto de datos ventas_nettalco

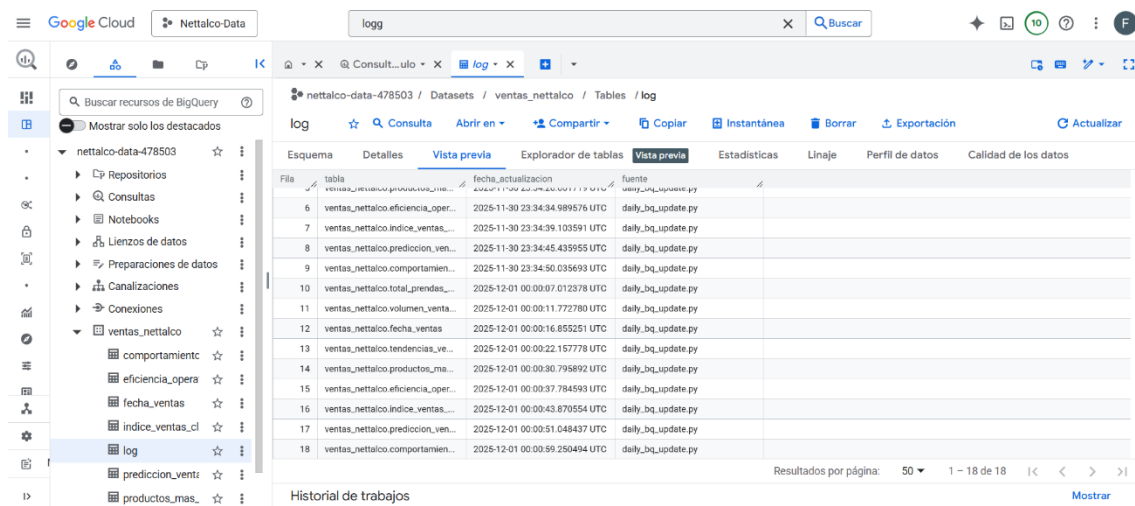
CREATE TABLE IF NOT EXISTS ventas_nettalco.log (

tabla STRING,

fecha_actualizacion TIMESTAMP,

fuelle STRING

);



6.6.Escalabilidad Horizontal

En cluster en dataproc cuenta con Autoscaling, implementadose con una política creada.

gcloud dataproc clusters update nettalco-cluster \

--region us-east1 \

--autoscaling-policy nettalo-autoscale

7. Scripts Utilizados

7.1. Script Principal PySpark

Archivo:

procesamiento_nettalco_etl.py

Funciones principales:

- Lectura desde GCS
- Transformación distribuida
- Escritura de resultados por dominio analítico

Ejemplos de métricas:

- Total de prendas por talla
- Ventas por cliente
- Eficiencia operativa
- Predicción con promedio móvil
- Comportamiento de clientes

8. Consultas SQL Analíticas

Las consultas SQL implementadas en BigQuery cumplen un rol fundamental dentro de la arquitectura analítica, ya que permiten validar la consistencia de los datos refinados, verificar los resultados de los procesos ETL ejecutados en Spark y habilitar el consumo analítico por herramientas de Business Intelligence.

Cada consulta fue diseñada para validar una tabla específica del Data Warehouse, garantizando integridad, coherencia y utilidad analítica de la información cargada desde la Zona Oro del Data Lake.

8.1. Validación de Volumen Total de Producción

SELECT

SUM(TOTAL_PRENDAS) AS total_prendas_suma

```
FROM `ventas_nettalco.total_prendas_por_talla`;
```

Objetivo técnico:

Verificar que la agregación global de producción coincida con los valores esperados tras el procesamiento distribuido.

Justificación:

Esta validación asegura que:

- No existen pérdidas de registros durante el ETL.
- Las agregaciones por talla fueron correctamente consolidadas.
- El volumen total de producción es consistente a nivel global.

8.2. Análisis de Clientes con Mayor Volumen de Ventas

```
SELECT
```

```
TCODICLIE,
```

```
TOTAL_PRENDAS
```

```
FROM `ventas_nettalco.volumen_ventas_por_cliente`
```

```
ORDER BY TOTAL_PRENDAS DESC
```

```
LIMIT 10;
```

Objetivo técnico:

Identificar los clientes con mayor impacto en el volumen total de ventas.

Justificación:

Permite:

- Validar la correcta agregación por cliente.
- Facilitar segmentación y análisis de rentabilidad.
- Proveer métricas clave para dashboards comerciales.

8.3. Validación por Franja Horaria

```
SELECT  
  
FRANJA_HORARIA,  
  
COUNT(*) AS registros,  
  
SUM(TOTAL_PRENDAS) AS total  
  
FROM `ventas_nettalco.tendencias_ventas_por_franja_horaria`  
  
GROUP BY FRANJA_HORARIA;
```

Objetivo técnico:

Verificar la correcta clasificación temporal de la producción.

Justificación:

Esta consulta confirma que:

- La lógica de clasificación por franja horaria aplicada en Spark es consistente.
- No existen registros sin asignación temporal.
- La distribución de producción por turnos es coherente.

8.4. Identificación de Productos Más Vendidos

```
SELECT  
  
ESTILO,  
  
TOTAL_PRENDAS  
  
FROM `ventas_nettalco.productos_mas_vendidos`  
  
ORDER BY TOTAL_PRENDAS DESC  
  
LIMIT 15;
```

Objetivo técnico:

Determinar los estilos con mayor demanda acumulada.

Justificación:

Esta información es clave para:

- Planeamiento de producción
- Análisis de portafolio de productos
- Validar la correcta agregación por dimensión producto

8.5. Validación de Eficiencia Operativa

SELECT

MIN(EFICIENCIA_PORCENTUAL) AS min_ef,

MAX(EFICIENCIA_PORCENTUAL) AS max_ef

FROM `ventas\_nettalco.eficiencia\_operativa`;

Objetivo técnico:

Evaluar los rangos extremos de eficiencia calculados durante el ETL.

Justificación:

Permite:

- Detectar valores atípicos
- Verificar que los porcentajes se encuentren en rangos válidos
- Asegurar coherencia de métricas operativas

8.6. Análisis de Tendencias con Promedio Móvil

SELECT

DATE(FECHA_TERMINO_TS) AS FECHA_TERMINO,

ESTILO,

TOTAL_PRENDAS,

PROMEDIO_MOVIL

FROM `ventas\_nettalco.prediccion\_ventas`

ORDER BY FECHA_TERMINO DESC

LIMIT 20;

Objetivo técnico:

Analizar tendencias temporales suavizadas mediante promedios móviles.

Justificación:

Esta consulta valida:

- La correcta implementación de funciones de ventana en Spark
- La estabilidad de tendencias frente a variabilidad diaria
- La preparación de los datos para análisis predictivo básico

8.7. Análisis de Comportamiento del Cliente

SELECT

TCODICLIE,

FRECUENCIA_COMPRA,

PROMEDIO_PRENDAS

FROM `ventas\_nettalco.comportamiento\_clientes`

ORDER BY FRECUENCIA_COMPRA DESC;

Objetivo técnico:

Caracterizar patrones de compra por cliente.

Justificación:

Permite:

- Identificar clientes recurrentes
- Validar métricas de frecuencia y volumen promedio
- Soportar estrategias de fidelización y segmentación

8.8. Rol de las Consultas SQL dentro de la Arquitectura

En conjunto, estas consultas cumplen tres funciones clave:

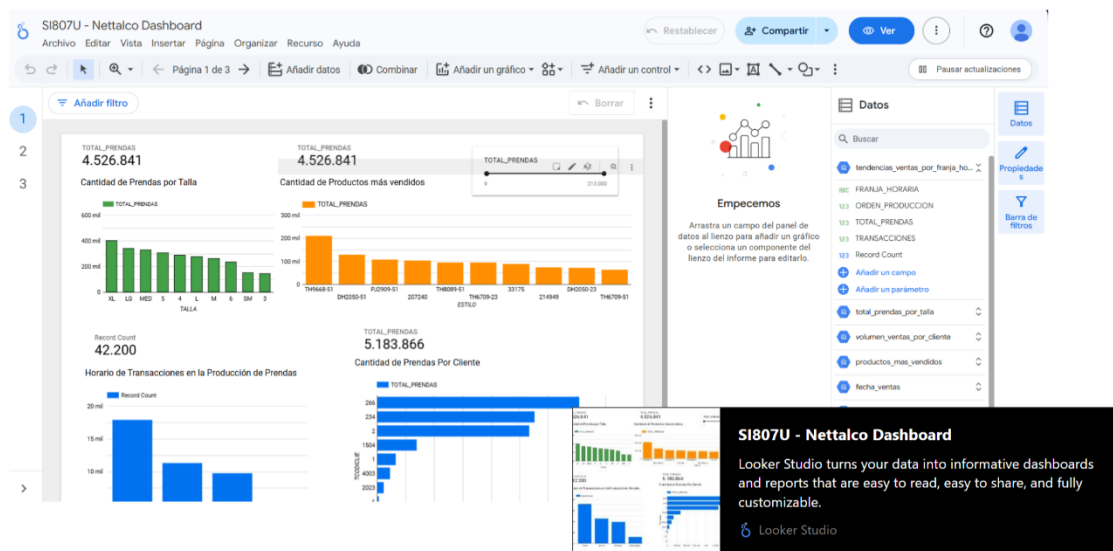
- Validación técnica del ETL
- Soporte al modelo dimensional
- Habilitación del consumo BI

De esta manera, BigQuery actúa como la capa de explotación analítica, mientras que la lógica compleja y pesada se mantiene en Spark, respetando el principio de separación de responsabilidades.

9. Visualizaciones Finales

9.1. Herramienta

Looker Studio – <https://lookerstudio.google.com/reporting/9139c4d1-2f52-4bd1-9e86-97b7554b2d58>



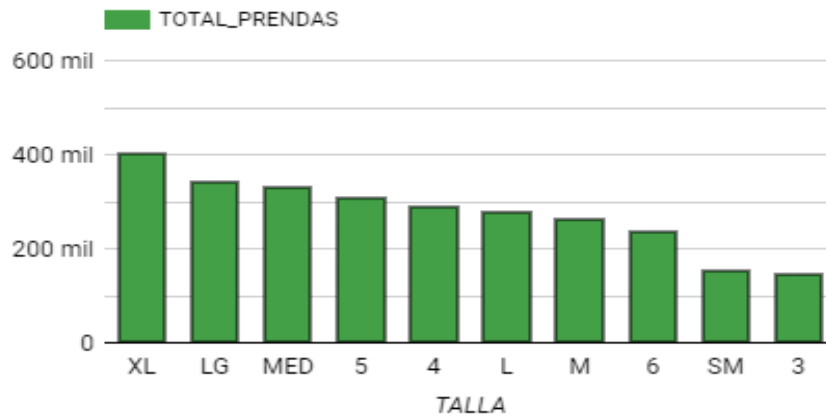
9.2. Dashboards Implementados

- Cantidad de prendas por talla

El enfoque de producción parece estar dirigido a las tallas más populares (XL, LG, MED), probablemente reflejando tendencias de mercado o preferencias del cliente. Es posible que las tallas más pequeñas sean nichos específicos o tengan una menor demanda.

TOTAL_PRENDAS
4.526.841

Cantidad de Prendas por Talla



- Cantidad de Productos más vendidos**

Este gráfico sugiere que algunos estilos tienen una aceptación mucho mayor en el mercado. Esto podría ser clave para decidir estrategias de marketing, producción, o incluso diseñar campañas de promoción para los estilos menos populares.



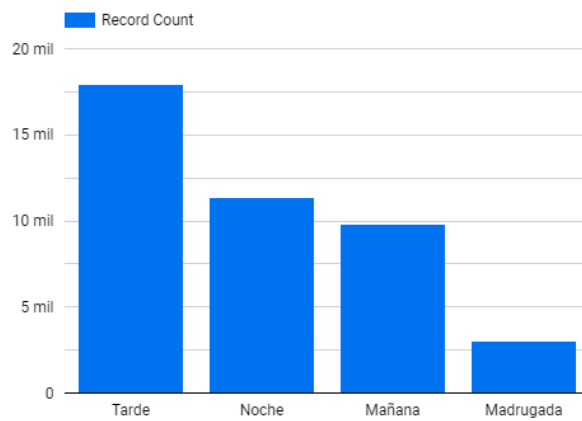
- Cantidad de transacciones realizadas**

Representa la cantidad de transacciones realizadas en diferentes franjas horarias del día. La tarde es el período con mayor actividad, superando las 15,000 transacciones, seguido por la noche y la mañana con valores más similares. La madrugada tiene significativamente menos transacciones.

Record Count

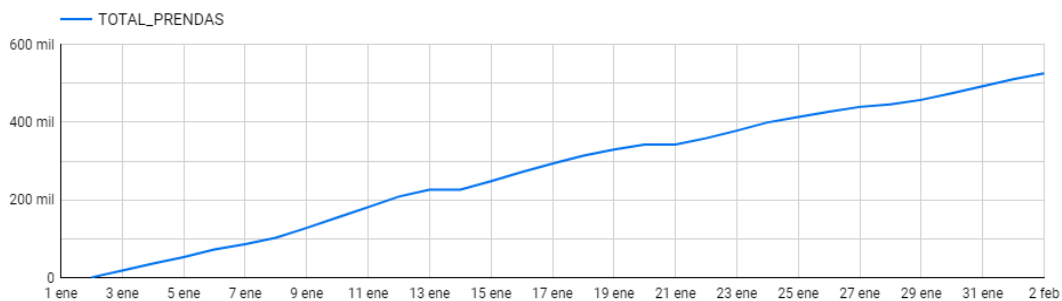
42.200

Horario de Transacciones en la Producción de Prendas



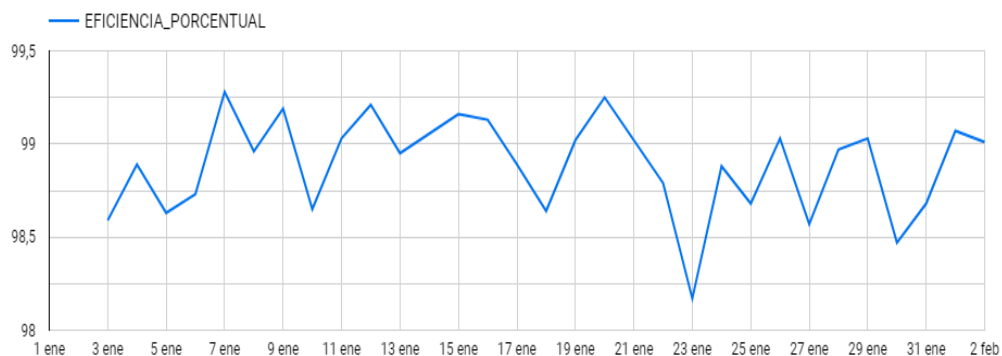
- **Total de prendas vendidas**

Muestra un crecimiento acumulativo en la producción de prendas durante el período del 1 de enero al 2 de febrero. Se observa un incremento constante, alcanzando cerca de 600,000 prendas.



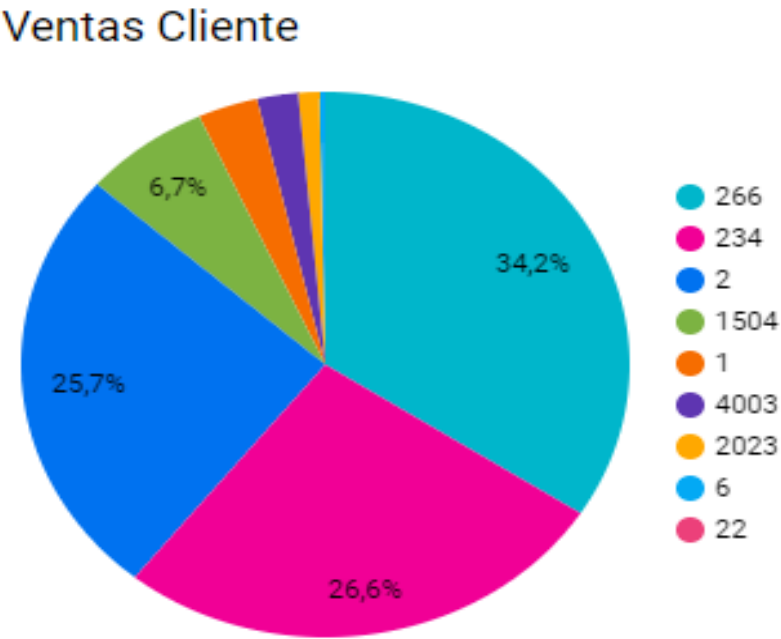
- **Eficiencia porcentual**

Representa la eficiencia diaria del proceso productivo durante el mismo período. Oscila entre el 98% y el 99.5%, mostrando estabilidad, aunque con pequeñas fluctuaciones.



- **Ventas de clientes nuevos**

Se puede apreciar que las ventas están altamente concentradas en dos clientes principales: el cliente 266, que representa el 34.2% de las ventas, y el cliente 234, con el 26.6%. Juntos, suman más del 60% del total, lo que destaca su importancia estratégica para el negocio. Esto indica la necesidad de fidelizar a estos clientes clave mientras se trabaja en diversificar los ingresos atrayendo y fortaleciendo relaciones con otros clientes.



- **Estadísticos de ventas por clientes**

los clientes 266 y 234 no solo tienen el mayor promedio de prendas compradas, sino también una alta frecuencia de compra, consolidándolos como los más valiosos. El cliente 1504 tiene una alta frecuencia de compra, aunque con un promedio menor de prendas, mientras que el cliente 4003 muestra un patrón opuesto con menos frecuencia pero un buen promedio. Esto sugiere oportunidades para incrementar la frecuencia de compra en clientes menos activos y aumentar el promedio de prendas en los más

frecuente

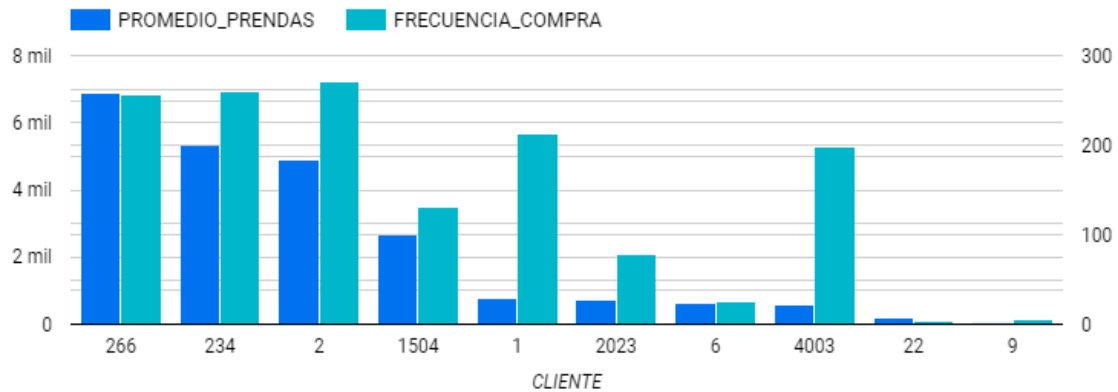
PROMEDIO_PRENDAS

22.730,35

FRECUENCIA_COMPRA

1.446

Comportamiento Clientes

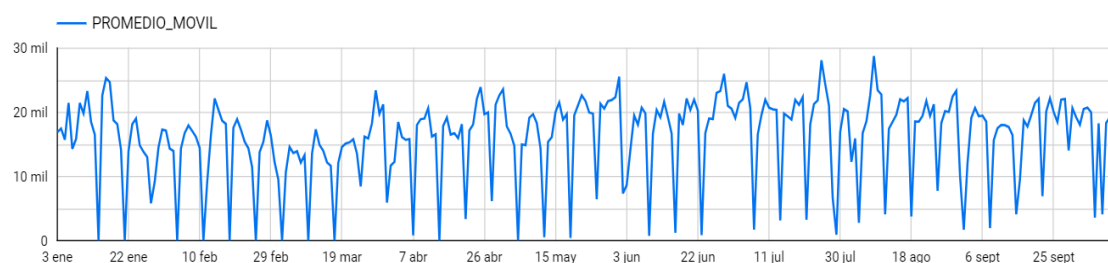


- **Predicción de ventas Nettelco**

Se observa en el grafico patrones recurrentes, con picos y caídas cíclicas que sugieren estacionalidad en las ventas.

Las ventas alcanzan máximos cercanos a los 30 mil y mínimos próximos a 0, lo que podría estar relacionado con días específicos o eventos. A lo largo del período, aunque existen fluctuaciones, la tendencia general parece estable, sin grandes incrementos o decrementos a largo plazo.

Predicción en Ventas



Justificación técnica:

- Conexión nativa con BigQuery
- No requiere infraestructura adicional
- Dashboards interactivos en tiempo real

10. Uso del Repositorio GitHub

10.1. Estrategia de Ramas

- main: versión estable
- feature/grupo05-pc3
- feature/grupo05-pc4
- feature/grupo05-ef

10.2. Pull Requests y Merges

- Desarrollo aislado por práctica (PC3,PC4 y EF)
- Revisión antes del merge
- Historial trazable de cambios

Justificación técnica:

- Control de versiones
- Reproducibilidad
- Trabajo colaborativo ordenado

11. Conclusión

La solución implementada cumple con los principios de una arquitectura Big Data moderna: escalabilidad, desacoplamiento, trazabilidad y optimización analítica. La integración de GCP, Spark, BigQuery y Looker permitió transformar datos operativos en información estratégica confiable, sentando las bases para análisis predictivo y toma de decisiones basada en datos.